

Book Recommendation RAG — Video Script

Sep 23, 2026 · @Context menu has integrated by the selected text, but into the b

3–4 minute walkthrough script for recording an internal explainer on the Book Recommendation RAG bot: stack, data, node-by-node architecture, LLM/embedding cost points, and a live multi-case demo.

Timeline

Time	Segment	Focus
0:00–0:20	Intro	What the bot does, tech stack
0:20–1:10	Data + design	Dataset, doc shape, why LangGraph state machine
1:10–2:20	Node walkthrough	understand → retrieveBooks → filterBooks → generateRecommendation, cost callouts
2:20–2:45	Query transition	Hardcoded → LLM-based prompt-to-query, why
2:45–3:50	Live demo	Genre+year, author fuzzy match, multi-turn follow-up, topic shift, out-of-scope
3:50–4:00	Close	Recap + next steps

Segment 1 — Intro & Framework (0:00–0:20)

On screen: repo in editor, `package.json` open.

Say: "This is a book recommendation bot built as one domain bot in a LangGraph multi-bot architecture. Stack is TypeScript end to end: LangGraph for orchestration as a state graph, Groq for LLM inference, Voyage AI for embeddings, and ChromaDB as the local vector store. Every piece — embedding provider, chat model — sits behind a small interface, so any vendor is swappable without touching callers."

Cue: point at `src/rag/embeddings.ts` (`EmbeddingProvider`) and `src/llm/model.ts` (`ChatModel`) briefly to show the swap seam.

Segment 2 — Data & System Design (0:20–1:10)

On screen: `data/raw/goodbooks-10k`, then `data/processed/books.cleaned.json` sample record.

Say (data, ~25s): "Data source is Goodbooks-10k — 10,000 books with ratings and tag data, no synopsis text. Raw CSVs get parsed, cleaned, and joined (`ingestion/cleanBooks.ts` , `mergeBookData.ts`) into one JSON record per book: title, authors, year, language, tags, rating. Each book becomes exactly one RAG document — `documents.ts` renders those fields into a short text block, so we're not chunking free text, we're embedding one structured record per book. That fits this dataset: it's metadata, not prose, so splitting further would just fragment an already-atomic unit."

Say (design rationale, ~25s): "Requirement was a conversational bot that handles both structured constraints — rating, year, language, author, genre — and semantic/mood intent, across multi-turn follow-ups that reference earlier turns. That's why it's modeled as a LangGraph state graph instead of a single prompt: each concern — understanding the request, retrieving, filtering, generating — is an isolated, testable node, and conversation memory is handled by LangGraph's own `MemorySaver` checkpointer keyed by `thread_id` , not hand-rolled state threading."

Segment 3 — Node Walkthrough & Cost Callouts (1:10–2:20)

On screen: `src/bots/book/graph.ts` graph diagram, then step through `nodes.ts` .

```
flowchart LR
    START --> understand
    understand -- out of scope --> END1[END]
    understand -- in scope --> retrieveBooks
    retrieveBooks --> filterBooks
    filterBooks --> generateRecommendation
    generateRecommendation --> END2[END]
```

Say, node by node (~70s total):

1. **understand** (~20s) — "One structured-output Groq call. It rewrites follow-ups into a standalone `resolvedQuery` using conversation history, checks if the request is even book-related (`inScope`), and extracts structured filters — rating, year, language, author, genre tags — all in one pass. **Cost: 1 LLM call per turn.** Out-of-scope short-circuits straight to END, no retrieval, no second LLM call."
2. **retrieveBooks** (~20s) — " `resolvedQuery` gets embedded via Voyage — **1 embedding call per turn** — then semantic search against Chroma, scoped to this bot's own `rag / domain` metadata and ANDed with a `where` clause built from the structured filters. If the user named an author, we pull a much bigger candidate pool (200 vs. the default top-K), because Chroma can't fuzzy-match author name formatting."

3. **filterBooks** (~15s) — "Client-side fuzzy author filter where Chroma's exact-match can't help, then a reranker seam — currently a no-op that just slices to top-K, ready to swap in a real cross-encoder or Voyage rerank later without touching callers."
4. **generateRecommendation** (~15s) — "Second and last Groq call. Strictly grounded on the retrieved candidates — the system prompt bans inventing books, authors, ratings, or descriptions. **Cost: 1 LLM call per turn.** Appends the turn to history for the next follow-up."

Cost summary line to say out loud: "So steady-state cost per user turn is 2 Groq calls plus 1 Voyage embedding call. Embedding the whole 10k-book dataset is a one-time ingestion cost, batched 128 documents per request."

Segment 4 — Prompt-to-Query Transition: Why LLM, Not Hardcoded (2:20–2:45)

On screen: `src/llm/prompts.ts` — `buildUnderstandingSystemPrompt`, then `src/bots/book/understanding.ts` fallback logic.

Say: "Early on, turning a user prompt into a retrieval query was handled with hardcoded rules — regex and keyword matching to detect follow-ups, decide scope, and pull out filters. That broke down fast: negation ('not fantasy' vs 'fantasy'), filler words, and topic shifts across turns each needed a new special case, and regex has no way to tell 'this follow-up still depends on the last answer' apart from 'this is actually a new topic' — that needs the model to actually read the conversation.

So that whole layer is now one LLM call — `understandNode` — that sees the last 4 turns of history plus the new message, and in one structured JSON response returns: the rewritten standalone query, an in/out-of-scope flag, and validated structured filters. It replaces three separate hardcoded mechanisms with one context-aware call, and it fails safe — if the response isn't valid JSON or fails schema validation, it falls back to treating the raw query as in-scope with no filters, so one bad completion never blocks a legitimate request."

Cue: show the `NOT fantasy` / topic-shift example from the prompt ("best spy book" after a non-fantasy list is a fresh request, not a search within that list) — concrete case of what regex couldn't do.

Segment 5 — Live Demo + Close (2:45–4:00)

On screen: terminal running `npm run dev`, same conversation thread throughout (shows `thread_id` memory working).

#	Prompt to type	What it demonstrates
1	"Recommend dystopian books published after 2008"	Genre tag + <code>publishedAfter</code> filter combined with semantic search
2	"Books by J.K. Rowling"	Author fuzzy match — 200-doc candidate pool, client-side case-insensitive substring filter in <code>filterBooksNode</code>
3	"What about rated above 4.5?" (same thread)	Multi-turn follow-up — <code>understand</code> rewrites this against history, REPLACES the old constraint rather than stacking it
4	"Best spy book" (same thread, after a non-fantasy list)	Topic shift — treated as a new independent request, not a search inside the earlier list
5	"Book me a flight to Delhi"	Out-of-scope — <code>inScope:false</code> , short-circuits straight to END, no retrieval/generation cost incurred

Say while running each: name what you expect before hitting enter, then point at the actual filters/answer that come back to confirm it matched.

Close (~10s): "That's the full loop — LangGraph nodes for orchestration, one LLM call to understand intent and history, semantic + metadata retrieval from Chroma, a second LLM call to generate a grounded answer. Next up: a real reranker and feedback-driven ranking, both already seamed in but not wired yet."